

Claim: E_{TM} is undecidable

→ No algo can determine whether any arbitrary TM accepts nothing.

Sipser Theorem 5.2, p. 217-218

$$E_{TM} = \{ \langle M \rangle \mid M \text{ is a TM and } L(M) \text{ is empty} \}$$

i.e. want to recognize codes of TMs that always reject / loop

$$E_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$$

M accepts no input at all.

For every possible input, it can reject / loop forever.

Proof by reduction ?

To use proof by reduction to prove that E_{TM} is undecidable, we must reduce an undecidable set to E_{TM}

$$A_{TM} \leq E_{TM}$$

If we could decide E_{TM} . Then we can also solve A_{TM} .

But A_{TM} is undecidable

$\therefore E_{TM}$ must also be undecidable.

Claim: E_{TM} is undecidable.

- **Proof by reduction**

- **Goal:** show that A_{TM} reduces to E_{TM} .

- **Assume:** have a TM, R , that decides E_{TM}

- **Build:** new TM, M_{ATM} , that decides A_{TM}

- Always halts
 - Accepts iff input $\langle M, w \rangle$ and w is in $L(M)$.



Claim: E_{TM} is undecidable.

- **Assume:** have a TM, R , that decides E_{TM}
- **Build:** new TM, M_{ATM} , that decides A_{TM}
 - Always halts
 - Accepts iff input $\langle M, w \rangle$ and w is in $L(M)$.



"On input $\langle M, w \rangle$:

- Run R on input $\langle M \rangle$. If rejects, reject.
- If accepts, run M on input w .
 - If accepts, accept; if reject, reject."

Claim: E_{TM} is undecidable.

- **Assume:** have a TM, R , that decides E_{TM}
- **Build:** new TM, M_{ATM} , that decides A_{TM}



- Always halts

- Accepts iff input $\langle M, w \rangle$ and w is in $L(M)$.

Does M accept no strings at all?

M accepts only 01 , $w = 01$.

Since $L(M) \neq \emptyset$, R reject.

But the proposed machine rejects even though $\langle M, w \rangle \in A_{TM}$

"On input $\langle M, w \rangle$:

- Run R on input $\langle M \rangle$. If rejects, reject.

$L(M) \neq \emptyset$
 M accepts some input.

- If accepts, run M on input w .

It does not mean that M reject w .

What is wrong here?

- If accepts, accept; if reject, reject."

In fact, M might accept w .

A_{TM}

M_{ATM} might not halt when $w \notin L(M)$. Not a decider.

Creating new TM in a reduction

$$A_{TM} \leq E_{TM}$$

$$\langle M, w \rangle \in A_{TM}$$

accepts.

R can only determine whether a machine's lang. is empty.

- Want to check if $M(w)$ ~~halts~~
- But, we can only check if $L(M)$ is empty or not
- Solution: **create new TM X** , hard-coding the inputs M, w .
- Then check if its language is empty or not

→ Convert to emptiness question, Construct new machine X with M and w permanently built inside it.

" $X(y)$:

- Run $M(w)$
- Return result"

Note that X ignores its own input!

→ Does $M(w)$: accept, reject or loop

Creating new TM in a reduction

"X(y):

- Run M(w)
- Return result"

A machine's lang. Contains only the string it accepts.
Rejecting and looping are diff. behaviour.
But neither one puts the input to the lang.

X accepts 0, 11, abc

M accepts w $\iff L(X) \neq \emptyset$

- What is the language of X?

(1) If M(w) accept, $L(X) = \Sigma^*$

(X accepts all inputs y)

Now give $\langle x \rangle$ to the assumed E_{TM} decider R:

(2) If M(w) rejects, $L(X) = \emptyset$

(X rejects all inputs y)

(3) If M(w) loops, $L(X) = \emptyset$

(X loops on all inputs y)

A_{TM} undecidable.

R cannot exist

E_{TM} undecidable.

If R says $L(X)$ is

not empty then

M accepted w.

R says $L(X) = \emptyset$
then M did not accept w.

Claim: E_{TM} is undecidable.

$$E_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$$

- **Assume:** have a TM, R , that decides E_{TM}
- **Build:** new TM, M_{ATM} , that decides A_{TM}
 - Always halts
 - Accepts iff input $\langle M, w \rangle$ and w is in $L(M)$.



R always halts.
 R accepts $\langle M \rangle$ if $L(M) = \emptyset$
 R rejects $\langle M \rangle$ if $L(M) \neq \emptyset$

Fixed version

"On input $\langle M, w \rangle$:

- First, build the TM X which, on input y , ignores y and simulates M on w .
- Run R on $\langle X \rangle$.
 - If accepts, reject; if rejects; accept."

If R accepts $\rightarrow L(X)$ is empty, so reject
If R rejects $\rightarrow L(X)$ is not empty, so accept

If M accepts w , X accepts γ .
If M rejects w , X rejects γ .
If M loops on w , X loops on γ .

Claim: E_{TM} is undecidable.

Fixed version

"On input $\langle M, w \rangle$:

- First, build the TM X which, on input y , ignores y and simulates M on w .
- Run R on $\langle X \rangle$.
 - If accepts, reject; if rejects, accept."

Does this machine always halt?

- ① Construct the description $\langle x \rangle$. This takes a finite amount of time.
- ② Run R on $\langle x \rangle$. By assumption, R is a decider, so it always halts.

Even when x would loop when executed, (not directly running x)
newly created M_{ATM} always halts.

Claim: E_{TM} is undecidable.

Fixed version

"On input $\langle M, w \rangle$:

- First, build the TM X which, on input y , ignores y and simulates M on w .
- Run R on $\langle X \rangle$.
 - If accepts, reject; if rejects; accept."

On input $\langle M, w \rangle$ where w is in $L(M)$...

$$w \in L(M)$$

Since $M(w)$, X accept every possible y .

$$\therefore L(X) = \Sigma^* \text{ (not empty)}$$

$R(\langle X \rangle) = \text{reject}$. (new machine reverses r answer so it accepts $\langle M, w \rangle$)
Thus $w \in L(M) \Rightarrow M_{TM}$ accepts.

Claim: E_{TM} is undecidable.

Fixed version

"On input $\langle M, w \rangle$:

- First, build the TM X which, on input y , ignores y and simulates M on w .
- Run R on $\langle X \rangle$.
 - If accepts, reject; if rejects; accept."

On input $\langle M, w \rangle$ where w is not in $L(M)$...

Case 1: M rejects w .

$$L(X) = \emptyset$$

Case 2: M loops on w .

$$L(X) = \emptyset$$

$R(\langle X \rangle)$ because $L(X)$ is empty.

Our machine reverses this and
and reject $\langle M, w \rangle$

Thus $w \notin L(M) \Rightarrow M_{ATM}$ rejects.

Generic framework

- Any “question” about the computation of a TM is undecidable, and often also un-recognizable
- You can prove it via reduction from A_{TM} or $HALT_{TM}$, sometimes using a new TM created for that

Decidable problems

$A_{NFA} = \{ \langle N, w \rangle \mid N \text{ is an NFA that accepts the string } w \}$

Prove A_{NFA} is decidable.

$A_{NFA} \rightarrow A_{DFA}$

We already know A_{DFA} is decidable: Simulate DFA D on w .

A DFA read one symbol at a time, so it always finishes at $|w|$ steps

For A_{NFA} :

- ① Convert NFA N into equivalent DFA.
- ② Run A_{DFA} decide on $\langle D, w \rangle$

Each proof checks two things:

- ① Halting: Every conversion is finite, final decider always stops.
- ② Correctness: The converted machine, accepts some string.

$A_{\text{rex}} = \{ \langle R, w \rangle \mid R \text{ is a regular exp that generates string } w \}$

$A_{\text{rex}} \rightarrow A_{\text{NFA}} \rightarrow A_{\text{DFA}}$

- ① Convert R into NFA N .
- ② Run A_{NFA} decider on $\langle N, w \rangle$